DIGITAL LOGIC AND COMPUTER DESIGN
UNIT 1 — DESCRIPTIVE QUESTIONS WITH ANSWERS

PES University | Descriptive Question Bank | T1: Harris & Harris, 2nd Edition

Q1. Define a digital circuit. Explain the functional specification and timing specification of a digital circuit. Distinguish between input, output and internal nodes.

Answer:
A digital circuit is a network of elements that processes discrete-valued variables, generally represented using binary values 0 and 1.

A digital circuit can be viewed as a black box having:
1. One or more input terminals
2. One or more output terminals
3. A functional specification
4. A timing specification

Functional specification describes the relationship between inputs and outputs. For example, for an AND function, Y = AB. The output is 1 only when both inputs are 1.

Timing specification describes how long the output takes to respond after an input changes. Thus, two circuits may implement the same Boolean function but have different delays.

A node is a wire that carries a digital signal. An input node receives information from outside the circuit, an output node sends information to another circuit/system, and an internal node connects elements inside the circuit.

Therefore, a digital circuit must be analyzed from both functional and timing perspectives.

Q2. A digital control system has three inputs A, B, C. The output Y must be HIGH whenever at least two of the inputs are HIGH. Obtain the truth table and derive the Boolean equation in sum-of-products form.

Answer:

Since the output is HIGH whenever at least two inputs are HIGH, the required input combinations are 011, 101, 110, and 111.

Truth table:

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

The corresponding minterms are:

m3 = A'BC

m5 = AB'C

m6 = ABC'

m7 = ABC

Therefore:

Y = A'BC + AB'C + ABC' + ABC

or

Y = Σ(3,5,6,7).

This demonstrates obtaining a Boolean equation directly from a truth table in canonical sum-of-products form.

Q3. For the Boolean function F(A,B,C) = Σ(1,3,5,7), obtain the canonical sum-of-products expression and simplify it using a K-map.

Answer:

The minterms are:

1 = 001

3 = 011

5 = 101

7 = 111

Hence the canonical SOP expression is:

F = A'B'C + A'BC + AB'C + ABC

Factor C:

F = C(A'B' + A'B + AB' + AB)

The expression in brackets contains all possible combinations of A and B:

A'B' + A'B + AB' + AB = 1

Therefore:

F = C

Hence:

F = C.

A K-map gives the same result by grouping the four 1s corresponding to C = 1.

Q4. Explain the difference between sum-of-products and product-of-sums canonical forms. Give a suitable example for each.

Answer:

In Sum-of-Products (SOP) form, the function is represented as an OR of product terms. Each product term corresponds to a row where the output is 1.

Example:
$F(A,B) = \Sigma(1,3)$
$F = A'B + AB$

In Product-of-Sums (POS) form, the function is represented as an AND of sum terms. Each maxterm corresponds to a row where the output is 0.

Example:
$F = \Pi(0,2)$
$F = (A+B)(A'+B)$

Difference:
SOP is an OR of AND terms and is based on output-1 rows (minterms). POS is an AND of OR terms and is based on output-0 rows (maxterms). SOP uses Σ notation, while POS uses Π notation.

Q5. A safety system produces output 1 for minterms 2, 3, 6, 7. Write the canonical SOP expression and explain how the resulting equation can be used to implement the circuit.

Answer:

For three variables A, B, C:
2 = 010
3 = 011
6 = 110
7 = 111

Therefore:
F = A'BC' + A'BC + ABC' + ABC

Factor B:
F = A'B(C'+C) + AB(C'+C)
Since C'+C = 1:
F = A'B + AB
F = B(A'+A)
F = B

Therefore:
F = B.

Implementation: instead of implementing four minterms using multiple gates, the minimized circuit can simply connect B to F. This reduces gate count, hardware complexity, and propagation delay.

Q6. A digital system has the following functional requirement: the output should be HIGH when the input combination is 0, 2, 4, or 6. Derive the Boolean function and explain how a K-map can be used to minimize it.

Answer:

Assume three inputs A, B, C.

The required minterms are:
$F = \Sigma(0,2,4,6)$

Canonical SOP:
F = A'B'C' + A'BC' + AB'C' + ABC'

Every selected minterm has C = 0. Therefore, on the K-map the four 1s form a group of four. Within this group A and B change, while C remains 0.

Thus:
F = C'

Therefore:
F = C'.

Q7. Explain the procedure for minimizing a Boolean function using a four-variable Karnaugh map. State the rules that must be followed while forming groups.

Answer:

A Karnaugh map is a graphical method for simplifying Boolean functions. A four-variable K-map contains 2^4 = 16 cells.

Procedure:

1. Draw the four-variable K-map.
2. Enter 1s corresponding to the specified minterms.
3. Enter X for don't-care conditions if given.
4. Identify adjacent 1s.
5. Form groups containing 1, 2, 4, 8, ... cells.
6. Make groups as large as possible.
7. Groups may wrap around the edges.
8. A cell may belong to more than one group when useful.
9. Obtain the implicant corresponding to each group.
10. OR all required implicants.

The objective is to obtain a minimal expression using the fewest/largest valid groups.

Q8. Minimize the following function using a four-variable K-map: F(A,B,C,D) = Σ(0,1,2,3,8,9,10,11).

Answer:

The minterms are 0, 1, 2, 3, 8, 9, 10, and 11. These eight cells form one group of eight.

Throughout this group:
B = 0
while A, C, and D vary.

Therefore the only remaining variable is B'.

Hence:
F = B'.

Q9. Minimize F(A,B,C,D) = Σ(1,3,5,7,9,11,13,15) using a K-map. Explain your grouping.

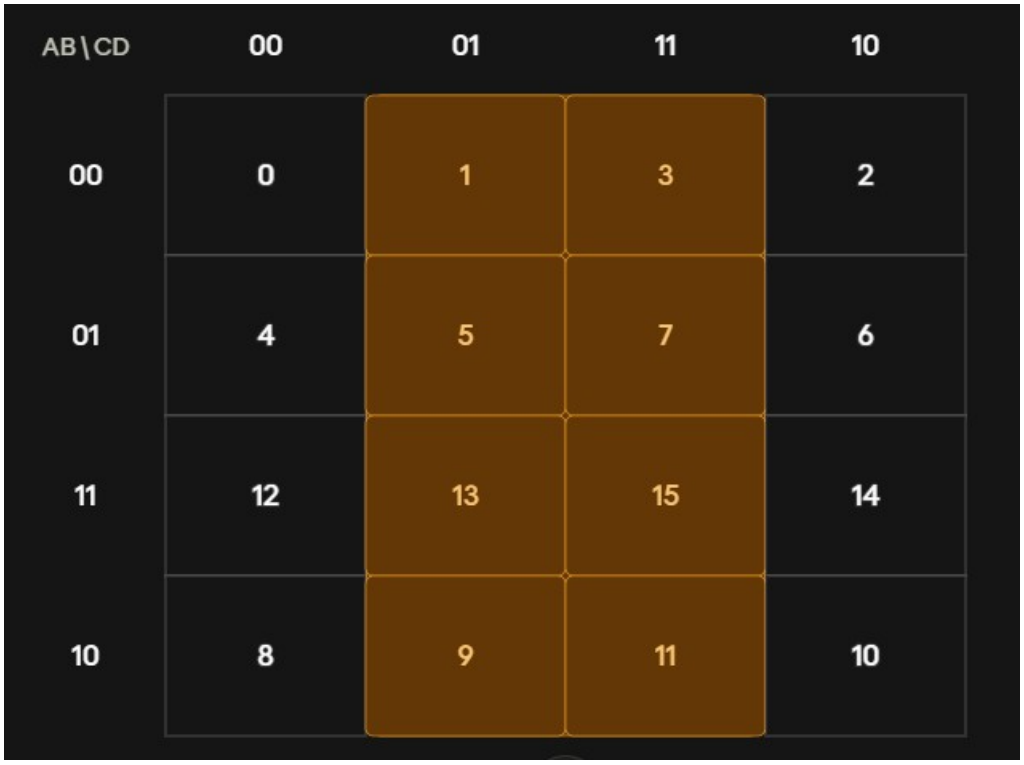
Answer:

All the specified minterms are odd. In binary, every odd number has D = 1.

Therefore, all eight cells corresponding to D = 1 can be grouped together. A, B, and C vary within the group, while D remains constant.

Hence:
F = D.

Thus the minimized function is:
F = D.



Q10. Minimize F(A,B,C,D) = Σ(0,2,8,10,11,14,15) using a four-variable K-map.

Answer:

Form the largest possible groups.

Group 1: minterms 0, 2, 8, 10 form a group of four. For this group B = 0 and D = 0, giving:
B'D'

Group 2: minterms 10, 11, 14, 15 form a group of four. For this group A = 1 and C = 1, giving:
AC

Therefore:
F = B'D' + AC.

The key point is to prefer maximum-sized valid groups so that the resulting expression contains fewer literals.



Q11. What are don't-care conditions? Explain how they help in K-map minimization.

Answer:

A don't-care condition occurs when the output of a circuit does not matter for a particular input combination. It is represented by X.

In a K-map, an X can be treated as either 0 or 1 depending on which choice produces a simpler expression.

For example, if a circuit uses only decimal inputs 0–9, inputs 10–15 are invalid. Their outputs can therefore be treated as don't-cares.

A don't-care should be included in a group when it helps form a larger group. If it does not help, it can be ignored.

Advantages include fewer product terms, fewer literals, fewer gates, and simpler hardware.

Q12. A seven-segment display receives a 4-bit input representing decimal digits 0–9. Inputs 10–15 are never used. Explain how these unused inputs can be exploited during K-map minimization.

Answer:

Since inputs 10–15 never occur during normal operation, their corresponding output values are irrelevant. They can be marked X in the K-map.

While grouping required 1s, these Xs may be included whenever they help produce a larger group. For example, a group of 2 may become a group of 4, or a group of 4 may become a group of 8.

Larger groups eliminate more variables, producing fewer gates and a simpler circuit.

Thus, unused input combinations can be exploited as don't-care conditions during K-map minimization.

Q13. Why is the minimal K-map expression not necessarily unique? Explain with reference to prime implicants.

Answer:

A Boolean function can sometimes be minimized in more than one valid way. This happens when a particular 1-cell can be included in more than one equally large prime implicant.

A prime implicant is an implicant that cannot be combined further to remove another literal.

If two different valid groupings cover all required 1s and each produces an expression with the same minimum cost, both expressions are minimal.

Therefore, the minimal expression of a Boolean function is not necessarily unique. What matters is that the expression is logically equivalent to the original function and meets the desired minimization criterion.

Q14. Explain the operation of a 2:1 multiplexer. Derive its Boolean equation and explain its truth table.

Answer:

A 2:1 multiplexer has two data inputs D0 and D1, one select input S, and one output Y.

Operation:
S = 0 -> Y = D0
S = 1 -> Y = D1

Therefore:
Y = S'D0 + SD1

When S = 0:
Y = 1·D0 + 0·D1 = D0

When S = 1:
Y = 0·D0 + 1·D1 = D1

Hence:
Y = S'D0 + SD1.

The multiplexer therefore selects one of several inputs and forwards the selected input to the output.

Q15. Explain how a 4:1 multiplexer can be constructed using 2:1 multiplexers. Determine the number of 2:1 multiplexers required.

Answer:

A 4:1 MUX has D0, D1, D2, D3 and two select lines S1 and S0.

First stage:
- MUX 1 selects between D0 and D1 using S0.
- MUX 2 selects between D2 and D3 using S0.

Second stage:
- MUX 3 selects between the outputs of MUX 1 and MUX 2 using S1.

Therefore:
3 two-to-one multiplexers are required.

The first stage performs two selections in parallel, while the second stage selects between the resulting two signals.

Q16. A processor needs to select one of eight possible data sources. Determine the type of multiplexer required and the number of select lines needed. Explain your answer.

Answer:

There are eight possible data inputs, so an 8:1 multiplexer is required.

For an N:1 multiplexer, the number of select lines is:
log2(N)

For N = 8:
log2(8) = 3

Therefore:
- Multiplexer required = 8:1
- Select lines required = 3

The three select bits provide 2^3 = 8 possible combinations, allowing each of the eight data sources to be selected.

Q17. Implement F(A,B,C) = Σ(1,2,5,7) using an 8:1 multiplexer.

Answer:

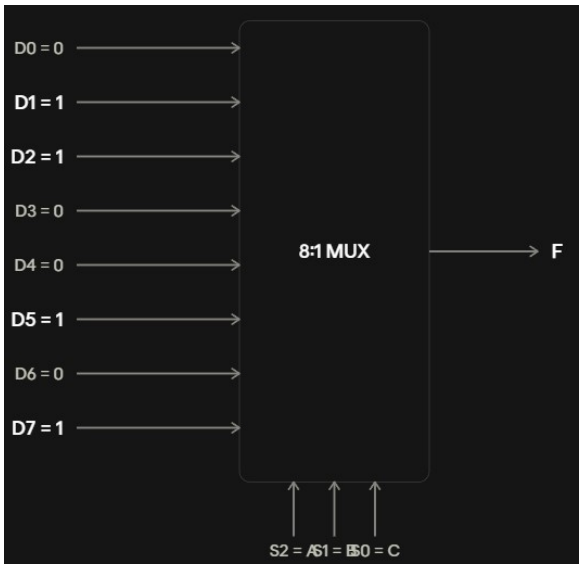
An 8:1 MUX has three select inputs. Choose:
S2 = A, S1 = B, S0 = C.

The data inputs correspond to minterms 0 through 7. Since the function is 1 for minterms 1, 2, 5, and 7:

D0 = 0
D1 = 1
D2 = 1
D3 = 0
D4 = 0
D5 = 1
D6 = 0
D7 = 1

Thus:
D0=0, D1=1, D2=1, D3=0, D4=0, D5=1, D6=0, D7=1.

With A, B, C connected to the select inputs, the MUX directly realizes the Boolean function.



Q18. Explain the operation of a decoder. How is a decoder different from a multiplexer?

Answer:

A decoder converts an N-bit binary input into one of up to 2^N outputs. For example, a 2-to-4 decoder has two inputs and four outputs. For each input combination, exactly one output is asserted.

Difference:
Multiplexer: many data inputs -> one output. It selects one input according to select lines.
Decoder: binary input -> one selected output. It identifies or activates an output according to the input code.

Thus, a MUX is primarily a data-selection circuit, whereas a decoder is primarily a code-to-one-of-many-output circuit.

Q19. A memory system contains eight memory modules. A 3-bit address is used to select exactly one module at a time. Explain how a decoder can be used for module selection.

Answer:

A 3-bit address has 2^3 = 8 possible combinations. Therefore, a 3:8 decoder can be used.

The three address bits are connected to the decoder inputs. Each decoder output corresponds to one memory module:

000 -> M0
001 -> M1
010 -> M2
011 -> M3
100 -> M4
101 -> M5
110 -> M6
111 -> M7

Only the selected decoder output is asserted, enabling the corresponding memory module. Thus, the decoder performs address decoding and ensures that one module is selected for each address.

Q20. Differentiate between combinational and sequential logic. Explain the role of memory/state in sequential circuits.

Answer:

In a combinational circuit, the output depends only on the current inputs:
$Y = f(X)$

There is no memory of previous inputs. Examples include multiplexers and decoders.

In sequential logic, the output depends on current inputs and previous state:
$Y = f(X,Q)$

where Q represents stored state.

The state stores information about past events. Therefore, sequential circuits can perform functions such as counting, storing data, and controlling a sequence of operations.

The key distinction is that combinational logic has no memory, whereas sequential logic has state/memory.

Q21. Given the input waveforms of an SR latch, explain how the output Q is determined. Discuss the SET, RESET and HOLD conditions.

Answer:

An SR latch has two inputs: S (Set) and R (Reset).

For an active-high SR latch:
S=0, R=0 -> HOLD
S=0, R=1 -> RESET
S=1, R=0 -> SET
S=1, R=1 -> invalid/forbidden

Set: when S=1 and R=0, Q becomes 1.
Reset: when S=0 and R=1, Q becomes 0.
Hold: when S=0 and R=0, the latch retains its previous state.

The important feature is memory: during HOLD, the current output depends on the previous state.

Q22. A D latch is transparent when CLK=1. If D during a HIGH clock period is 0 -> 1 -> 0 -> 1, explain the output sequence. What happens when the clock becomes LOW?

Answer:

A D latch is level-sensitive. When CLK=1, it is transparent, so Q follows D.

Therefore, during the HIGH clock period:
D: 0 -> 1 -> 0 -> 1
Q: 0 -> 1 -> 0 -> 1

When CLK becomes 0, the latch becomes opaque and holds its last value.

Therefore, after the clock goes LOW:
Q = 1

Even if D subsequently changes while CLK remains LOW, Q does not change until the latch becomes transparent again.

Q23. Compare a D latch and a positive-edge-triggered D flip-flop for the same changing input signal. Explain why the two devices can produce different outputs.

Answer:

A D latch is level-sensitive. When its active clock level is present, changes in D can propagate to Q.

A positive-edge-triggered D flip-flop is edge-sensitive. It copies D to Q only at the rising clock edge.

If D changes several times while CLK is HIGH:
- A D latch can respond to all those changes.
- A positive-edge-triggered D flip-flop captures only the value present at the rising edge.

Therefore, the two devices can produce different output waveforms for the same input sequence.

This difference is fundamental: a latch is transparent during an active level, while a flip-flop samples at a clock edge.

Q24. Construct a T flip-flop using a D flip-flop and logic. Derive the required D input equation.

Answer:

A T flip-flop toggles when T=1 and holds its state when T=0.

Truth table:

T Q | Q+
0 0 | 0
0 1 | 1
1 0 | 1
1 1 | 0

Therefore:

$Q^+ = T \text{ XOR } Q$

For a D flip-flop:

$Q^+ = D$

Hence:

$D = T \text{ XOR } Q$

So the implementation is:

1. Feed T and the current Q into an XOR gate.
2. Connect the XOR output to D.
3. Use the same clock for the D flip-flop.

Thus:

$D = T \oplus Q$.

Q25. A JK flip-flop is to be implemented using a D flip-flop. Derive the expression for D in terms of J, K and Q.

Answer:

The JK flip-flop has:
J=0, K=0 -> Hold
J=0, K=1 -> Reset
J=1, K=0 -> Set
J=1, K=1 -> Toggle

Its characteristic equation is:
Q+ = JQ' + K'Q

For a D flip-flop:
Q+ = D

Therefore:
D = JQ' + K'Q

Hence, a D flip-flop can implement a JK flip-flop by connecting the output of combinational logic implementing JQ' + K'Q to D.

Q26. Explain the operation of an N-bit register. Why are multiple flip-flops required?

Answer:

An N-bit register stores an N-bit binary value. It is constructed using N flip-flops, normally D flip-flops, with a common clock.

For a 4-bit register:
Inputs: D3, D2, D1, D0
Outputs: Q3, Q2, Q1, Q0

At the active clock edge, all flip-flops capture their corresponding inputs simultaneously.

Therefore:
Q3:0+ = D3:0

Multiple flip-flops are required because each flip-flop stores one bit. Thus N bits require N storage elements.

A register is therefore a bank of flip-flops sharing a common clock.

Q27. A processor register should load new data only when ENABLE is HIGH. When ENABLE is LOW, it must retain its previous value even when clock edges occur. Explain how an enabled flip-flop achieves this requirement.

Answer:

An enabled flip-flop has D, CLK, EN, and Q.

Its operation is:
EN=1 -> at the active clock edge, Q loads D.
EN=0 -> at the active clock edge, Q retains its old value.

Therefore:
Q+ = D, when EN=1
Q+ = Q, when EN=0

A MUX can be placed before the D input:
DFF = EN·D + EN'·Q

When EN=1:
DFF = D

When EN=0:
DFF = Q

Thus the old output is fed back and retained when the register is disabled.

Q28. A digital system requires a flip-flop to return to a known state whenever RESET is asserted. Explain the purpose of a resettable flip-flop and distinguish between resetting the state and loading normal data.

Answer:

A resettable flip-flop contains an additional RESET input. When RESET is asserted, the flip-flop is forced to a known state, generally Q=0.

This is important because sequential circuits need a known starting state.

During normal operation, when RESET is inactive, the flip-flop captures D at the active clock edge:
Q+ = D

When RESET is asserted:
Q -> 0

Thus, RESET is used for initialization or recovery to a known state, while D is used for normal data storage.

Q29. A ring oscillator consists of three identical inverters connected in a loop. Each inverter has a propagation delay of 2 ns. Determine the oscillation period and frequency. Explain why the circuit is not a stable combinational circuit.

Answer:

There are N=3 inverters and each has propagation delay tp=2 ns.

For a ring oscillator:
T = 2Ntp

Therefore:
T = 2(3)(2 ns)
T = 12 ns

Frequency:
f = 1/T
= 1/(12 × 10^-9)
≈ 83.33 MHz

The circuit is not a stable combinational circuit because the output of each inverter is fed back into the input of another. The feedback prevents a stable Boolean assignment and causes the signal to continually propagate around the loop, producing oscillation.

Q30. An elevator controller is designed for a building containing 25 floors, but floor 13 does not exist. The controller must remember the current floor. Determine the minimum number of state bits required and explain your reasoning.

Answer:

The controller must represent 25 valid floor positions.

The number of state bits N must satisfy:
$2^N >= 25$

For 4 bits:
$2^4 = 16 < 25$

Therefore 4 bits are insufficient.

For 5 bits:
$2^5 = 32 >= 25$

Hence:
N = 5 state bits

Therefore, at least 5 bits of state are required to uniquely represent all 25 valid floor positions. The unused state encodings can be treated as invalid/unreachable states and handled appropriately in the controller design.